

Android 动态权限（Lazy 模式）实现

Android 运行时动态权限（Lazy 模式）详解：Java + XML 实现

作为 Android 小白，你可能遇到过这样的问题：明明在 AndroidManifest.xml 中声明了权限，运行时却报错“权限被拒绝”？这是因为 **Android 6.0（API 23）及以上**，系统将权限分为“普通权限”和“危险权限”，危险权限需要在**运行时动态向用户申请**，而不能只靠清单文件声明。

今天我们重点讲解 **Lazy 模式（懒加载模式）** 的动态权限申请——核心思想是：**用到某个权限时才申请**（比如调用相机时才申请相机权限，访问位置时才申请定位权限），而不是 App 启动时一次性申请所有权限，这样能减少对用户的打扰，提升用户体验。

一、先搞懂 3 个核心概念

1. 危险权限 vs 普通权限

- 普通权限**：不涉及用户隐私、不影响系统安全（如网络访问、震动权限），只需在 AndroidManifest.xml 声明即可，系统自动授予。
- 危险权限**：涉及用户隐私或系统安全（如相机、存储、定位、麦克风），必须在运行时动态申请，用户手动同意后才能使用。

✔ 常见危险权限（需要动态申请的）：

权限组	具体权限（示例）
CAMERA	相机权限 (android.permission.CAMERA)
STORAGE	读写存储 (android.permission.READ_EXTERNAL_STORAGE、 WRITE_EXTERNAL_STORAGE)
LOCATION	定位权限 (android.permission.ACCESS_FINE_LOCATION)
RECORD_AUDIO	

录音权限 (android.permission.RECORD_AUDIO)

2. Lazy 模式的核心逻辑

1. **触发条件：**当用户执行需要危险权限的操作时（如点击“拍照”按钮），才触发权限申请。
2. **申请流程：**
 - 检查权限是否已授予 → 已授予则直接执行操作；
 - 未授予则向系统申请权限 → 用户同意则执行操作；
 - 用户拒绝则提示“需要该权限才能使用功能”，并引导用户去设置中开启。
3. **优势：**避免 App 启动时弹出多个权限申请弹窗，提升用户体验。

3. 关键 API (Java)

- 检查权限：`ContextCompat.checkSelfPermission(Context, 权限字符串)` → 返回 `PackageManager.PERMISSION_GRANTED`（已授予）或 `PERMISSION_DENIED`（未授予）。
- 申请权限：`ActivityCompat.requestPermissions(Activity, 权限数组, 请求码)` → 向系统弹出权限申请弹窗。
- 处理权限申请结果：`onRequestPermissionsResult(int requestCode, String[] permissions, int[] grantResults)` → 监听用户的同意/拒绝操作。
- 检查是否需要解释权限：
`ActivityCompat.shouldShowRequestPermissionRationale(Activity, 权限)`
→ 当用户第一次拒绝权限后，再次申请时返回 `true`（可向用户解释为什么需要该权限）。

二、实战案例：拍照功能（动态申请相机权限）

我们以“点击按钮拍照”为例，完整实现 Lazy 模式的动态权限申请，包含 XML 布局和 Java 逻辑。

步骤 1：在 AndroidManifest.xml 中声明权限（必做）

无论是否动态申请，危险权限都需要先在清单文件中声明（否则申请会失败）：

Code block

```
1  <!-- 声明相机权限 -->
2  <uses-permission android:name="android.permission.CAMERA" />
3  <!-- 可选：如果拍照后需要保存图片到存储，还需声明存储权限 -->
4  <uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"
5      android:maxSdkVersion="28" /> <!-- 安卓10及以上用分区存储，无需此
    权限 -->
```

步骤 2: XML 布局（简单的拍照按钮）

创建 `activity_main.xml`，添加一个按钮用于触发拍照操作：

Code block

```
1  <?xml version="1.0" encoding="utf-8"?>
2  <LinearLayout
3      xmlns:android="http://schemas.android.com/apk/res/android"
4      android:layout_width="match_parent"
5      android:layout_height="match_parent"
6      android:gravity="center"
7      android:orientation="vertical">
8
9      <!-- 拍照按钮：点击时触发权限申请和拍照逻辑 -->
10     <Button
11         android:id="@+id/btn_take_photo"
12         android:layout_width="wrap_content"
13         android:layout_height="wrap_content"
14         android:text="点击拍照"
15         android:textSize="18sp" />
16
17 </LinearLayout>
```

步骤 3: Java 逻辑（核心：Lazy 模式权限申请）

创建 `MainActivity.java`，实现“点击按钮 → 检查权限 → 申请权限 → 处理结果 → 执行拍照”的完整流程：

Code block

```
1  import android.Manifest;
2  import android.content.pm.PackageManager;
3  import android.os.Bundle;
4  import android.view.View;
5  import android.widget.Button;
6  import android.widget.Toast;
7
8  import androidx.annotation.NonNull;
9  import androidx.appcompat.app.AppCompatActivity;
10 import androidx.core.app.ActivityCompat;
11 import androidx.core.content.ContextCompat;
12
13 public class MainActivity extends AppCompatActivity {
14
```

```
15 // 1. 定义权限相关常量 (请求码: 用于识别权限申请结果)
16 private static final int REQUEST_CAMERA_PERMISSION = 1001; // 自定义, 唯一即可
17 private Button btnTakePhoto;
18
19 @Override
20 protected void onCreate(Bundle savedInstanceState) {
21     super.onCreate(savedInstanceState);
22     setContentView(R.layout.activity_main);
23
24     // 2. 绑定按钮, 设置点击事件 (触发拍照操作)
25     btnTakePhoto = findViewById(R.id.btn_take_photo);
26     btnTakePhoto.setOnClickListener(new View.OnClickListener() {
27         @Override
28         public void onClick(View v) {
29             // 3. 点击按钮后, 先检查相机权限 (Lazy 模式核心: 用到时才检查)
30             checkCameraPermission();
31         }
32     });
33 }
34
35 // 4. 检查相机权限的逻辑
36 private void checkCameraPermission() {
37     // 检查当前是否已授予相机权限
38     int permissionStatus = ContextCompat.checkSelfPermission(
39         this,
40         Manifest.permission.CAMERA // 要检查的权限
41     );
42
43     if (permissionStatus == PackageManager.PERMISSION_GRANTED) {
44         // 情况1: 权限已授予 → 直接执行拍照操作
45         takePhoto();
46     } else {
47         // 情况2: 权限未授予 → 申请权限
48         requestCameraPermission();
49     }
50 }
51
52 // 5. 向系统申请相机权限
53 private void requestCameraPermission() {
54     // 检查是否需要向用户解释“为什么需要该权限” (用户第一次拒绝后, 再次申请时触发)
55     boolean needExplain =
56     ActivityCompat.shouldShowRequestPermissionRationale(
57         this,
58         Manifest.permission.CAMERA
59     );
```

```

60         if (needExplain) {
61             // 向用户解释：“需要相机权限才能拍照，请允许”
62             Toast.makeText(this, "请授予相机权限，否则无法拍照",
Toast.LENGTH_SHORT).show();
63         }
64
65         // 向系统弹出权限申请弹窗（参数：当前Activity、权限数组、请求码）
66         ActivityCompat.requestPermissions(
67             this,
68             new String[]{Manifest.permission.CAMERA}, // 要申请的权限（可多
个，数组形式）
69             REQUEST_CAMERA_PERMISSION // 请求码，后续用于识别结果
70         );
71     }
72
73     // 6. 处理权限申请结果（系统回调方法，必须重写）
74     @Override
75     public void onRequestPermissionsResult(
76         int requestCode, // 之前传入的请求码
77         @NonNull String[] permissions, // 申请的权限数组
78         @NonNull int[] grantResults // 权限申请结果数组（与permissions一一对
应）
79     ) {
80         super.onRequestPermissionsResult(requestCode, permissions,
grantResults);
81
82         // 只处理我们的相机权限由请结果里（避免混淆其他权限由请）

```

三、关键细节解析（小白必看）

1. 请求码（requestCode）的作用

- 必须是**唯一的整数**（如 1001、2002），用于区分多个权限申请（比如同时申请相机和存储权限时，通过请求码判断哪个权限的申请结果）。
- 建议用 `private static final` 定义，避免魔法数字。

2. 权限数组的使用（申请多个权限）

如果需要同时申请多个权限（如拍照+存储），只需修改权限数组：

Code block

```

1 // 同时申请相机和存储权限
2 ActivityCompat.requestPermissions(
3     this,
4     new String[]{
5         Manifest.permission.CAMERA,

```

```
6             Manifest.permission.WRITE_EXTERNAL_STORAGE
7         },
8         REQUEST_CAMERA_STORAGE_PERMISSION // 新的请求码
9     );
```

处理结果时，需要遍历 `grantResults` 数组，检查每个权限是否都被授予。

3. `shouldShowRequestPermissionRationale` 的作用

- 当用户**第一次拒绝**权限后，再次申请时，该方法返回 `true` → 可向用户解释权限的用途（比如“需要存储权限才能保存照片”）。
- 当用户拒绝并勾选“不再询问”后，该方法返回 `false` → 此时无法再通过代码申请权限，只能引导用户去系统设置中手动开启。

4. 安卓 10+（API 29）的存储权限特殊处理

- 安卓 10 及以上引入“分区存储”，`WRITE_EXTERNAL_STORAGE` 权限失效，无需申请，直接通过 `MediaStore` API 保存文件即可。
- 如果需要兼容旧版本，可在清单文件中添加 `android:maxSdkVersion="28"`（如步骤 1 所示）。

四、Lazy 模式 vs 启动时申请（对比分析）

模式	适用场景	优点	缺点
Lazy 模式 （用到时申请）	大部分 App（如拍照、定位、录音功能）	1. 减少用户打扰；2. 权限申请与功能强关联，用户更容易理解；3. 提升 App 启动速度	1. 首次使用功能时能打断用户操作；需要权限的功能编辑
启动时申请 （一次性申请所有权限）	权限依赖强的 App（如地图 App 必须要定位权限）	1. 一次性完成所有权限申请，后续使用功能无弹窗；2. 逻辑简单，无需在多个地方检查权限	1. 启动时弹窗过多直接拒绝所有权限 App 启动体验

✅ 结论：优先使用 **Lazy 模式**，符合 Android 官方推荐的权限设计规范，提升用户体验。

五、常见问题排查（小白避坑）

1. 申请权限时无弹窗，直接返回拒绝？

- 检查清单文件是否声明了该权限（步骤 1 必做）。

- 检查权限是否属于“危险权限”（普通权限无需动态申请）。
- 检查设备系统版本是否 $\geq$ Android 6.0（API 23）（低于 6.0 的系统，清单声明即可授予）。

2. 用户拒绝后，无法再次申请权限？

- 可能是用户勾选了“不再询问”，此时 `shouldShowRequestPermissionRationale` 返回 `false`，需引导用户去设置中开启：

Code block

```
1 // 当用户拒绝且不再询问时，引导至设置
2 Toast.makeText(this, "请在系统设置中开启相机权限", Toast.LENGTH_SHORT).show();
3 Intent intent = new Intent(Settings.ACTION_APPLICATION_DETAILS_SETTINGS);
4 Uri uri = Uri.fromParts("package", getPackageName(), null);
5 intent.setData(uri);
6 startActivity(intent);
```

3. 权限申请结果回调不执行？

- 确保 `onRequestPermissionsResult` 方法是**重写**（`@Override`）的，没有拼写错误。
- 确保申请权限时的 `requestCode` 与回调中的 `requestCode` 一致。

六、扩展：封装权限申请工具类（优化代码）

如果 App 中有多个地方需要申请权限，可封装一个工具类，避免重复代码。示例：

Code block

```
1 // 权限申请工具类
2 public class PermissionUtils {
3     // 检查权限是否已授予
4     public static boolean hasPermission(Context context, String permission) {
5         return ContextCompat.checkSelfPermission(context, permission)
6             == PackageManager.PERMISSION_GRANTED;
7     }
8
9     // 申请权限
10    public static void requestPermission(Activity activity, String[]
permissions, int requestCode) {
11        ActivityCompat.requestPermissions(activity, permissions, requestCode);
12    }
13
14    // 检查是否需要解释权限
15    public static boolean needExplainPermission(Activity activity, String
permission) {
```

```
16         return ActivityCompat.shouldShowRequestPermissionRationale(activity,  
17         permission);  
18     }
```

使用时简化代码：

Code block

```
1  // 检查相机权限（简化后）  
2  if (PermissionUtils.hasPermission(this, Manifest.permission.CAMERA)) {  
3      takePhoto();  
4  } else {  
5      PermissionUtils.requestPermission(this, new String[]  
6      {Manifest.permission.CAMERA}, REQUEST_CAMERA_PERMISSION);  
7  }
```

总结

Lazy 模式的动态权限申请核心是“**用到时才申请**”，流程可总结为 3 步：

1. 执行需要权限的操作前，检查权限是否已授予；
2. 未授予则向系统申请，必要时向用户解释权限用途；
3. 监听权限申请结果，同意则执行操作，拒绝则提示用户。

通过本文的案例（拍照功能），你可以直接复用代码到自己的项目中，只需替换权限字符串（如 `Manifest.permission.LOCATION`）和具体操作逻辑（如 `takePhoto()` 替换为 `getLocation()`）即可。

如果需要进一步学习“多个权限同时申请”“引导用户去设置开启权限”等进阶内容，可以随时告诉我！

（注：文档部分内容可能由 AI 生成）